CORS Vulnerability with Trusted Null Origin

Lab Overview

- **Platform:** PortSwigger Web Security Academy
- **Vulnerability:** Cross-Origin Resource Sharing (CORS)
- **Difficulty:** Practitioner-level lab
- **Category:** Web Application Security

1. Description

The `Origin: null` value can appear in requests from certain contexts, including sandboxed documents and some local-file scenarios. Browsers may serialize the origin as `null` when the requesting document has an opaque origin.

A CORS vulnerability occurs when a server explicitly trusts the `null` origin and grants it access to sensitive resources. An attacker may be able to use a suitable browser context to make cross-origin requests and read responses that should remain protected.

2. Vulnerability Identification

I examined the application's CORS behavior using Burp Suite and tested how the server handled a request with a `null` origin.

Example request:

GET /accountDetails HTTP/1.1

Host: target.example

Origin: null

Cookie: session=USER_SESSION

Example vulnerable response:

HTTP/1.1 200 OK

Access-Control-Allow-Origin: null

Access-Control-Allow-Credentials: true

Content-Type: application/json

The response indicates that the server explicitly trusts the `null` origin and permits credentialed cross-origin access. This can be dangerous if the endpoint returns sensitive information.

Note: These HTTP examples are illustrative. Replace them with the actual request and response from the lab before publishing your report.

3. Testing Methodology

1. Launched the PortSwigger lab and identified a potentially sensitive endpoint.
2. Intercepted the request using Burp Suite.
3. Modified the `Origin` header to `null`.
4. Sent the request and inspected the response headers.
5. Checked whether the server returned `Access-Control-Allow-Origin: null`.
6. Reviewed the credential policy and assessed whether a browser context with an opaque origin could read the response.

4. Security Impact

If the vulnerable endpoint exposes sensitive information and the browser sends the victim's credentials, an attacker may be able to retrieve account details or other protected data through a malicious webpage.

The presence of `Access-Control-Allow-Origin: null` alone does not prove successful exploitation. The endpoint's contents, credential handling, browser restrictions, and the attacker's ability to create a suitable context must also be considered.

5. Remediation

- Avoid trusting the `null` origin for sensitive endpoints.
- Use an explicit allowlist of trusted origins.
- Do not enable credentialed cross-origin access unless required.
- Minimize sensitive information returned by API endpoints.
- Validate CORS behavior across different browser-origin contexts.
- Include negative tests for `null`, arbitrary, and untrusted origins.

6. Key Takeaways

This lab demonstrated the security risks of trusting the `null` origin in a CORS policy. Secure origin validation should be explicit and restrictive, particularly when endpoints return authenticated or sensitive information.

Tools Used: Burp Suite, browser developer tools, PortSwigger Web Security Academy.